

Performance Estimation Problems (PEPs)

Systematic analysis and design of first-order optimization algorithms
via the performance estimation framework

François Glineur, Adrien Taylor



SIAM Conference on Optimization, tutorial session – June 2026



| One-slide summary

Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ (continuously differentiable) with $f \in \mathcal{F}$. Find $x_\star \in \mathbb{R}^d$ such that

$$f(x_\star) = \min_{x \in \mathbb{R}^d} f(x).$$

For example, assume ($x_\star = 0$, $f_\star = 0$ for readability):

| One-slide summary

Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ (continuously differentiable) with $f \in \mathcal{F}$. Find $x_\star \in \mathbb{R}^d$ such that

$$f(x_\star) = \min_{x \in \mathbb{R}^d} f(x).$$

For example, assume ($x_\star = 0$, $f_\star = 0$ for readability):

◇ Gradient-based method, e.g.: $x_k = x_{k-1} - \sum_{i=0}^{k-1} \gamma_{k,i} \nabla f(x_i)$,

| One-slide summary

Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ (continuously differentiable) with $f \in \mathcal{F}$. Find $x_\star \in \mathbb{R}^d$ such that

$$f(x_\star) = \min_{x \in \mathbb{R}^d} f(x).$$

For example, assume ($x_\star = 0$, $f_\star = 0$ for readability):

- ◇ Gradient-based method, e.g.: $x_k = x_{k-1} - \sum_{i=0}^{k-1} \gamma_{k,i} \nabla f(x_i)$,
- ◇ $\mathcal{F}$ can be described through nice linear/quadratic (interpolation) inequalities

| One-slide summary

Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ (continuously differentiable) with $f \in \mathcal{F}$. Find $x_\star \in \mathbb{R}^d$ such that

$$f(x_\star) = \min_{x \in \mathbb{R}^d} f(x).$$

For example, assume ($x_\star = 0$, $f_\star = 0$ for readability):

- ◇ Gradient-based method, e.g.: $x_k = x_{k-1} - \sum_{i=0}^{k-1} \gamma_{k,i} \nabla f(x_i)$,
- ◇ $\mathcal{F}$ can be described through nice linear/quadratic (interpolation) inequalities in terms of $F = [f_0, \dots, f_n]$ and $P = [x_0, \nabla f(x_0), \dots, \nabla f(x_n)]$ of the form $\text{Tr}(P^T P A_i) + b_i^T F \leq 0$.

| One-slide summary

Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ (continuously differentiable) with $f \in \mathcal{F}$. Find $x_\star \in \mathbb{R}^d$ such that

$$f(x_\star) = \min_{x \in \mathbb{R}^d} f(x).$$

For example, assume ($x_\star = 0$, $f_\star = 0$ for readability):

- ◇ Gradient-based method, e.g.: $x_k = x_{k-1} - \sum_{i=0}^{k-1} \gamma_{k,i} \nabla f(x_i)$,
- ◇ $\mathcal{F}$ can be described through nice linear/quadratic (interpolation) inequalities in terms of $F = [f_0, \dots, f_n]$ and $P = [x_0, \nabla f(x_0), \dots, \nabla f(x_n)]$ of the form $\text{Tr}(P^T P A_i) + b_i^T F \leq 0$.
- ◇ Performance criterion/initial condition linear in $(G = P^T P, F)$.

| One-slide summary

Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ (continuously differentiable) with $f \in \mathcal{F}$. Find $x_\star \in \mathbb{R}^d$ such that

$$f(x_\star) = \min_{x \in \mathbb{R}^d} f(x).$$

For example, assume ($x_\star = 0$, $f_\star = 0$ for readability):

- ◇ Gradient-based method, e.g.: $x_k = x_{k-1} - \sum_{i=0}^{k-1} \gamma_{k,i} \nabla f(x_i)$,
- ◇ $\mathcal{F}$ can be described through nice linear/quadratic (interpolation) inequalities in terms of $F = [f_0, \dots, f_n]$ and $P = [x_0, \nabla f(x_0), \dots, \nabla f(x_n)]$ of the form $\text{Tr}(P^T P A_i) + b_i^T F \leq 0$.
- ◇ Performance criterion/initial condition linear in $(G = P^T P, F)$.

Then:

- ◇ problem of computing worst-case performance is an SDP,
- ◇ proofs of the fact “Perf $\leq \gamma$ Init”

| One-slide summary

Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ (continuously differentiable) with $f \in \mathcal{F}$. Find $x_\star \in \mathbb{R}^d$ such that

$$f(x_\star) = \min_{x \in \mathbb{R}^d} f(x).$$

For example, assume ($x_\star = 0$, $f_\star = 0$ for readability):

- ◇ Gradient-based method, e.g.: $x_k = x_{k-1} - \sum_{i=0}^{k-1} \gamma_{k,i} \nabla f(x_i)$,
- ◇ $\mathcal{F}$ can be described through nice linear/quadratic (interpolation) inequalities in terms of $F = [f_0, \dots, f_n]$ and $P = [x_0, \nabla f(x_0), \dots, \nabla f(x_n)]$ of the form $\text{Tr}(P^T P A_i) + b_i^T F \leq 0$.
- ◇ Performance criterion/initial condition linear in $(G = P^T P, F)$.

Then:

- ◇ problem of computing worst-case performance is an SDP,
- ◇ proofs of the fact “Perf $\leq \tau$ Init” can be framed as: $\exists (\lambda_i)_i$, $\lambda_i \geq 0$, and $S \succeq 0$

$$0 \geq \sum_i \lambda_i (\text{Tr}(G A_i) + b_i^T F) = \text{Perf} - \tau \text{Init} + \text{Tr}(G S).$$

| When does it work?

- ◇ Function classes: described by quadratic inequalities.

| When does it work?

- ◇ Function classes: described by quadratic inequalities.
- ◇ Composite problem structures,

$$\min_{x \in \mathbb{R}^d} \sum_{i=1} f_i(x)$$

→ exploited in, e.g., splitting and stochastic methods.

| When does it work?

- ◇ Function classes: described by quadratic inequalities.
- ◇ Composite problem structures,

$$\min_{x \in \mathbb{R}^d} \sum_{i=1} f_i(x)$$

→ exploited in, e.g., splitting and stochastic methods.

- ◇ Compositions with linear operator: $f(Mx) \rightarrow$ exploited, e.g., in primal-dual methods.

| When does it work?

- ◇ Function classes: described by quadratic inequalities.
- ◇ Composite problem structures,

$$\min_{x \in \mathbb{R}^d} \sum_{i=1} f_i(x)$$

→ exploited in, e.g., splitting and stochastic methods.

- ◇ Compositions with linear operator: $f(Mx) \rightarrow$ exploited, e.g., in primal-dual methods.
- ◇ Algorithms:

| When does it work?

- ◇ Function classes: described by quadratic inequalities.
- ◇ Composite problem structures,

$$\min_{x \in \mathbb{R}^d} \sum_{i=1} f_i(x)$$

→ exploited in, e.g., splitting and stochastic methods.

- ◇ Compositions with linear operator: $f(Mx) \rightarrow$ exploited, e.g., in primal-dual methods.
- ◇ Algorithms:
 - Gradient and *inexact* gradient-based (e.g., $\|d - \nabla f(x)\|^2 \leq \epsilon$) methods,

| When does it work?

- ◇ Function classes: described by quadratic inequalities.
- ◇ Composite problem structures,

$$\min_{x \in \mathbb{R}^d} \sum_{i=1} f_i(x)$$

→ exploited in, e.g., splitting and stochastic methods.

- ◇ Compositions with linear operator: $f(Mx) \rightarrow$ exploited, e.g., in primal-dual methods.
- ◇ Algorithms:
 - Gradient and *inexact* gradient-based (e.g., $\|d - \nabla f(x)\|^2 \leq \epsilon$) methods,
 - Proximal/projection methods (implicit gradients): $x_{k+1} = x_k - \gamma g_{k+1}$,

| When does it work?

- ◇ Function classes: described by quadratic inequalities.
- ◇ Composite problem structures,

$$\min_{x \in \mathbb{R}^d} \sum_{i=1} f_i(x)$$

→ exploited in, e.g., splitting and stochastic methods.

- ◇ Compositions with linear operator: $f(Mx) \rightarrow$ exploited, e.g., in primal-dual methods.
- ◇ Algorithms:
 - Gradient and *inexact* gradient-based (e.g., $\|d - \nabla f(x)\|^2 \leq \epsilon$) methods,
 - Proximal/projection methods (implicit gradients): $x_{k+1} = x_k - \gamma g_{k+1}$,
 - Linear optimization oracles: $y = \operatorname{argmin}_{x \in \mathcal{C}} \langle c, x \rangle \Leftrightarrow -c \in \partial i_{\mathcal{C}}(y)$,
 - etc.

| When does it work?

- ◇ Function classes: described by quadratic inequalities.
- ◇ Composite problem structures,

$$\min_{x \in \mathbb{R}^d} \sum_{i=1} f_i(x)$$

→ exploited in, e.g., splitting and stochastic methods.

- ◇ Compositions with linear operator: $f(Mx) \rightarrow$ exploited, e.g., in primal-dual methods.
- ◇ Algorithms:
 - Gradient and *inexact* gradient-based (e.g., $\|d - \nabla f(x)\|^2 \leq \epsilon$) methods,
 - Proximal/projection methods (implicit gradients): $x_{k+1} = x_k - \gamma g_{k+1}$,
 - Linear optimization oracles: $y = \operatorname{argmin}_{x \in \mathcal{C}} \langle c, x \rangle \Leftrightarrow -c \in \partial i_{\mathcal{C}}(y)$,
 - etc.
- ◇ Beyond optimization: fixed-point setup, monotone inclusions, etc.

| When does it work?

- ◇ Function classes: described by quadratic inequalities.
- ◇ Composite problem structures,

$$\min_{x \in \mathbb{R}^d} \sum_{i=1} f_i(x)$$

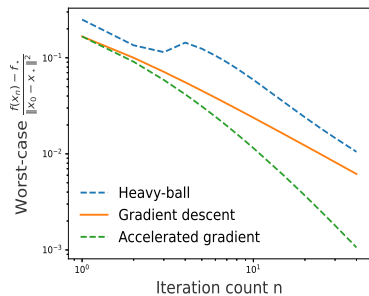
→ exploited in, e.g., splitting and stochastic methods.

- ◇ Compositions with linear operator: $f(Mx) \rightarrow$ exploited, e.g., in primal-dual methods.
- ◇ Algorithms:
 - Gradient and *inexact* gradient-based (e.g., $\|d - \nabla f(x)\|^2 \leq \epsilon$) methods,
 - Proximal/projection methods (implicit gradients): $x_{k+1} = x_k - \gamma g_{k+1}$,
 - Linear optimization oracles: $y = \operatorname{argmin}_{x \in \mathcal{C}} \langle c, x \rangle \Leftrightarrow -c \in \partial i_{\mathcal{C}}(y)$,
 - etc.
- ◇ Beyond optimization: fixed-point setup, monotone inclusions, etc.

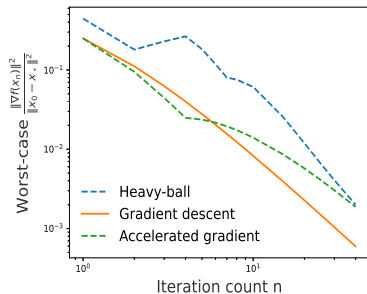
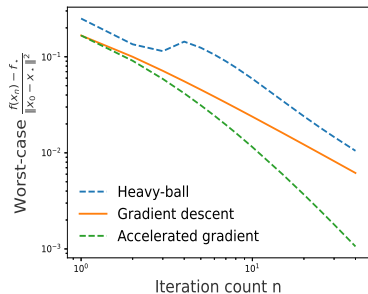
List of working examples: <https://pepit.readthedocs.io/>

| Typical output from a numerical (vanilla) PEP?

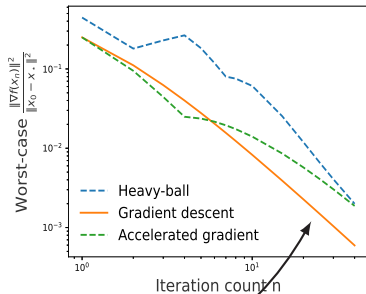
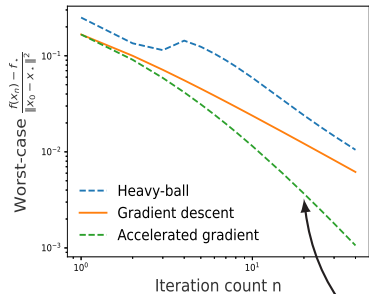
| Typical output from a numerical (vanilla) PEP?



| Typical output from a numerical (vanilla) PEP?



| Typical output from a numerical (vanilla) PEP?



What to expect
for larger n ?

| Recap'

| Recap'

- 😊 Worst-case guarantees *cannot be improved*, systematic approach,
- 😊 allows reaching analyses that could barely be obtained by hand,
- 😊 fair amount of scenarios/algorithms (e.g., proximal terms, stochastic, etc.),
- 😞 SDPs typically become prohibitively large in a variety of scenarios,
- 😞 transient behavior VS. asymptotic behavior: might be hard to distinguish with small N ,
- 😞 proofs (may be) quite involved and hard to intuit,
- 😞 proofs (may be) hard to generalize.

A few open-ended directions

Directions (by different groups):

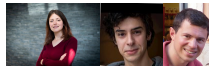
- ◇ applications beyond “pure” optimization (fixed-point, monotone inclusions, etc.),
- ◇ “structured” problems (less black-box, refined analyses),
- ◇ structured proofs (restrict the proof search space: Lyapunov functions, etc.),
- ◇ study of adaptative algorithms,
- ◇ study of higher-order methods,
- ◇ algorithm design,
- ◇ symbolic solves & formalization.

| A few (random?) instructive examples — algorithms & analyses from PEPs

- ◇ Drori, Teboulle ('14). "Performance of first-order methods for smooth convex minimization: a novel approach."
- ◇ Kim, Fessler ('16). "Optimized methods for smooth convex optimization."
- ◇ Van Scoy, Freeman, Lynch ('17). "The fastest known globally convergent first-order method for minimizing strongly convex functions."
- ◇ Kim, Fessler ('21). "Optimizing the efficiency of first-order methods for decreasing the gradient of smooth convex functions."
- ◇ Altschuler, Parrilo ('23). "Acceleration by Stepsize Hedging: Multi-Step Descent and the Silver Stepsize Schedule."
- ◇ Grimmer ('24). "Provably faster gradient descent via long steps."
- ◇ Jang, Das Gupta, Ryu ('25). "Computer-assisted design of accelerated composite optimization methods: OptISTA."
- ◇ Nilsson, Akerman, Giselsson ('25). "Extending Douglas-Rachford Splitting for Convex Optimization."

| Going further — shameless advertisement (Francois')

- ◇ Bouselmi, Hendrickx, G ('24). "Interpolation conditions for linear operators and applications to performance estimation problems"
- ◇ Kamri, Hendrickx, G ('25). "Numerical Design of Optimized First-Order Algorithms."
- ◇ Zamani, Glineur ('25). "Exact convergence rate of the last iterate in subgradient methods."
- ◇ Rubbens, Bouselmi, Hendrickx, G ('25). "Performance Estimation of second-order optimization methods on classes of univariate functions."
- ◇ Rotaru, G, Patrinos ('26). "Exact worst-case convergence rates of gradient descent: a complete analysis for all constant stepsizes over nonconvex and convex functions."

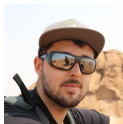


| Going further — shameless advertisement (Adrien's)

- ◇ Upadhyaya, Banert, T, Giselsson ('25). "Automated tight Lyapunov analysis for first-order methods."
- ◇ Berg Thomsen, T, Dieuleveut ('25). "Tight analyses of first-order methods with error feedback."
- ◇ Rubbens, Hendrickx, T ('25). "A constructive approach to strengthen algebraic descriptions of function and operator classes."
- ◇ Weibel, Gaillard, Koolen, T ('26). "Optimised projection-free algorithms for online learning: construction and worst-case analysis."
- ◇ Naldi, El Din, T, Wang ('26). "Solving parametric linear matrix inequalities."



Software/material/thoughts built with many great colleagues, among which:



Baptiste
Goujaud



Céline
Moucher



Julien
Hendrickx



Aymeric
Dieuleveut



Shuvomoy
Das Gupta

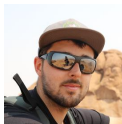


Daniel
Berg Thomsen



Weijia
Wang

Software/material/thoughts built with many great colleagues, among which:



Baptiste
Goujaud



Céline
Moucher



Julien
Hendrickx



Aymeric
Dieuleveut



Shuvomoy
Das Gupta



Daniel
Berg Thomsen



Weijia
Wang

Material & packages: community work, please don't hesitate to feedback/contribute

The screenshot shows the GitHub repository for PEPit. At the top, it says "PEPit" with a "Public" label. Below this are buttons for "Edit Pins", "Unwatch" (5), "Fork" (18), and "Starred" (111). The repository has a "master" branch, 8 branches, and 11 tags. A search bar "Go to file" is present. Below the repository name, there is a merge pull request #141 from PerformanceEstimation/fix/mino... with a green checkmark, commit 5a061a1, 2 weeks ago, and 1,203 commits. A table lists files: ".github/workflows" with the description "python 3.13 version in tests" (5 months ago) and "PEPit" with the description "fixed docstring" (2 weeks ago). On the right, the "About" section states: "PEPit is a package enabling computer-assisted worst-case analyses of first-order optimization methods." and provides a link to pepit.readthedocs.io/en/latest/.

... and ★ them on Github if you liked!

Thank you for your attention!

Last wooclap time!

[https://app.wooclap.com/THIKNZU?from=
instruction-slide](https://app.wooclap.com/THIKNZU?from=instruction-slide)

